

Exame de Paradigmas de Programação — Época de Recurso / Modelo 4

|||

|---|---|

Instituição	P.PORTO — Escola Superior de Tecnologia e Gestão
Tipo de Prova	Exame Escrito — Época de Recurso / Modelo 4
Curso	Licenciatura em Engenharia Informática / Licenciatura em Segurança Informática em Redes de Computadores
Unidade Curricular	Paradigmas de Programação
Ano Letivo	2025/2026
Duração	2 horas

Observações

- Não é permitida a consulta.
- Não são permitidas questões relativas à Parte 2. Sempre que considerarem necessário, os alunos devem assumir os pressupostos que entenderem adequados, indicando-os explicitamente na resolução.

Parte 1

Pergunta 1

1,5 val

Explique o conceito de **Ligação Dinâmica** (*Dynamic Binding* ou *Late Binding*) em Java e como esta viabiliza o Polimorfismo. Como é que a Máquina Virtual de Java (JVM) determina em tempo de execução qual o método concreto a ser executado quando a invocação é feita através de uma referência de uma superclasse ou interface?

Pergunta 2

1,5 val

O que é o conceito de **Imutabilidade** em Orientação a Objetos? Quais são as vantagens de desenhar classes imutáveis (por exemplo, no contexto de segurança e concorrência)? Liste e explique as regras necessárias para criar uma classe completamente imutável em Java.

Pergunta 3

1,5 val

No desenvolvimento de aplicações robustas em Java, é frequente a criação de **Exceções Personalizadas** (*Custom Exceptions*). Explique como se cria uma classe de exceção personalizada herdando de `Exception` ou `RuntimeException`. Qual a importância de invocar o construtor da superclasse (`super(...)`) passando a mensagem de erro e a causa (*cause*)?

Pergunta 4

1,5 val

Explique a utilização do operador `instanceof` e do mecanismo de conversão de tipos (*downcasting*). Quais os riscos de realizar um *downcasting* sem validação prévia e que exceção pode ser lançada pela JVM? Ilustre a forma correta e segura de efetuar a conversão com um exemplo prático em Java.

Parte 2

1. Considere as seguintes interfaces `Container` e `MedicineContainer`. A interface `Container` descreve um contendor genérico. A interface `MedicineContainer` especializa `Container` para armazenar medicamentos que exigem um histórico de medições de temperatura e controlo rígido de tolerância térmica.

```
public interface Container {  
    String getCode();  
    ItemType getType();  
    double getCapacity();  
}
```

```
public interface MedicineContainer extends Container {  
    double getTargetTemperature();  
    Measurement[] getMeasurementHistory();  
    void addMeasurement(Measurement measurement) throws ContainerException;  
    boolean hasTemperatureViolation(double tolerance);  
    boolean equals(Object obj);  
}
```

Pergunta 1a

3 val

Implemente a interface `MedicineContainer` numa classe denominada `MedicineContainerImpl`.

Regras de Implementação:

- A classe deve armazenar as medições de temperatura num array interno com capacidade para até 100 medições.
- O método `addMeasurement(Measurement m)` deve adicionar a medição ao histórico. Caso o histórico já esteja cheio (100 medições), deve lançar uma `ContainerException`.
- O método `hasTemperatureViolation(double tolerance)` deve verificar se alguma medição guardada no histórico tem um valor que se desvie da temperatura alvo (`getTargetTemperature()`) em mais do

que o valor de `tolerance` (ou seja, se `|valor - targetTemperature| > tolerance`).

- Duas instâncias de `MedicineContainer` são consideradas iguais se possuírem o mesmo código (`getCode()`).

Pergunta 1b

2 val

Desenvolva o código necessário para testar a classe `MedicineContainerImpl` num método `main`. O teste deve instanciar um contentor de medicamentos, adicionar medições com e sem violação térmica, e demonstrar o funcionamento de `hasTemperatureViolation` e de `equals`.

2. Considere a existência de uma classe `QualityControlImpl` que implementa a interface `QualityControl`.

```
public interface QualityControl {  
    boolean isAidBoxSafe(AidBox aidbox, double tempTolerance);  
    MedicineContainer[] getCompromisedContainers(IIInstitution inst, double tempTolerance);  
    QualityReport generateReport(IIInstitution inst, double tempTolerance);  
}
```

Pergunta 2a

4 val

Na classe `QualityControlImpl`, implemente os seguintes métodos auxiliares:

```
boolean isAidBoxSafe(AidBox aidbox, double tempTolerance);
```

- Este método verifica se uma `AidBox` está em condições de segurança no que diz respeito aos medicamentos.
- Percorre todos os contentores da `AidBox`. Se algum contentor for instância de `MedicineContainer` e apresentar uma violação de temperatura (utilizando `hasTemperatureViolation(tempTolerance)`), a caixa deixa de ser segura e o método deve retornar `false`.
- Caso nenhum contentor de medicamentos esteja comprometido, retorna `true`.

```
MedicineContainer[] getCompromisedContainers(IIInstitution inst, double tempTolerance);
```

- Este método percorre todas as `AidBoxes` de `inst.getAidBoxes()`.
- Identifica todos os `MedicineContainer` presentes nas caixas que apresentem violação de temperatura.
- Retorna um array perfeitamente dimensionado (sem posições nulas) contendo **todos** os `MedicineContainer` comprometidos identificados na instituição.

Pergunta 2b

5 val

Na classe `QualityControlImpl`, implemente o método `generateReport`:

```
QualityReport generateReport(IInstitution inst, double tempTolerance);
```

Regras a considerar:

- Contabilize o total de contentores de medicamentos (`MedicineContainer`) existentes em toda a instituição.
- Utilize o método `getCompromisedContainers` para obter os contentores comprometidos.
- Calcule a taxa de integridade do sistema como o número de contentores de medicamentos **não comprometidos** a dividir pelo **total** de contentores de medicamentos (retornando `1.0` se não existirem contentores de medicamentos).
- Crie e devolva uma instância da classe `QualityReportImpl`, passando no construtor: o total de contentores de medicamentos analisados, a contagem de comprometidos e a taxa de integridade calculada.

Excertos de Código Fornecidos

```
public interface IInstitution {
    AidBox[] getAidBoxes();
}

public interface AidBox {
    Container[] getContainers();
}

public interface Container {
    ItemType getType();
}

public interface Measurement {
    double getValue();
}

public interface QualityReport {
    int getTotalMedicineContainers();
    int getCompromisedContainersCount();
    double getIntegrityRate();
}
```